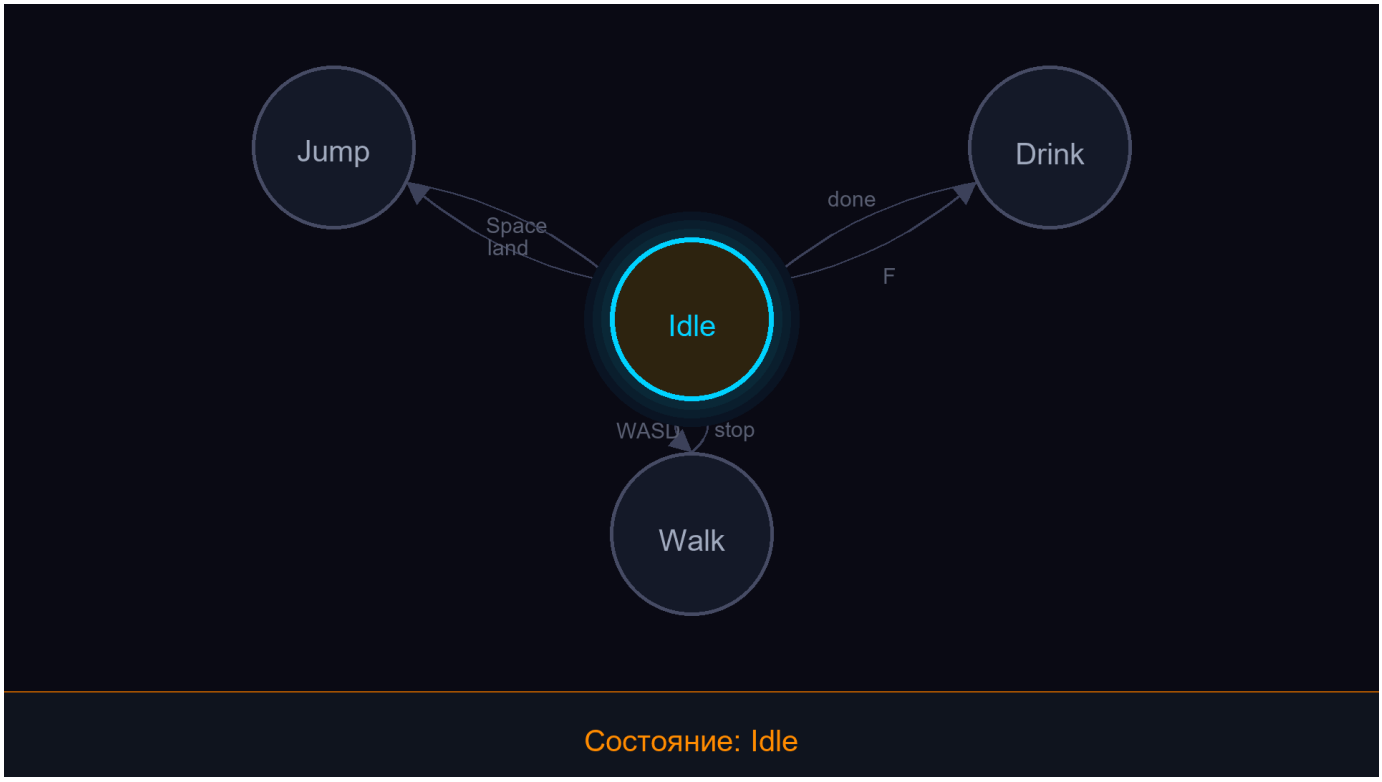
Конечные автоматы

Конечные автоматы

И теперь плавно переходим от архитектуры к алгоритмам. Начнем с State Machine. С конечных автоматов. Мы не будем рассматривать полноценную теорию автоматов, все их виды и правила работы с ними. Тут нам потребуется следующие идеи:

- **Абстрактный автомат** - некая абстракция, модель устройства, схожая с математической функцией - что-то подается на вход, как-то перерабатывается и получается результат.
- **Конечный автомат** - это абстрактный автомат, который находится в одном конкретном состоянии и может менять его в зависимости от каких-то входящих данных.

Для понимания желательно вспомнить графы



Давайте возьмем в качестве примера передвижение человечка по карте. Как мы можем описать ситуацию, когда человечек ничего не делает? Состояние покоя. Когда человек прыгает и находится в воздухе? Состояние прыжка, он прыгнул и до сих пор не приземлился. Не все действия доступны в состоянии прыжка. Воду пить не хотелось бы.

Начальное состояние: `человечек ничего не делает`. Если `нажать Space`, он подпрыгнет. Если `нажать F`, то он начнет пить воду. Но если он уже находится в прыжке, то `нажатие F` не должно приводить к выпиванию воды (с точки зрения геймдизайна такое возможно, но не всегда нужно). Более того, во время прыжка мы не хотим открывать двери.

Представляете как эти требования могут выглядеть в виде кода? Примерно так:

Python C#

```
is_jumping = False
is_drinking = False

#...
while running:
    for event in pygame.event.get():
        #...
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_SPACE and not is_jumping and not is_d
                is_jumping = True  # Character starts jumping
            elif event.key == pygame.K_f and not is_jumping and not is_dri
                is_drinking = True  # Character starts drinking water

    # ...
    # Draw character
    if is_jumping:
        pygame.draw.rect(screen, BLUE, (player_pos[0], player_pos[1] - 20,
        is_jumping = False
    elif is_drinking:
        pygame.draw.ellipse(screen, RED, (player_pos[0], player_pos[1], pla
        is_drinking = False
    else:
        pygame.draw.rect(screen, BLUE, (player_pos[0], player_pos[1], playe
```

А теперь добавьте дополнительные условия, особенно в духе *"если в прыжке, но только начали, то можно еще отменить прыжок, а если оторвались от земли, то уже не можем отменить"*. Ну, получается примерно так:

Python C#

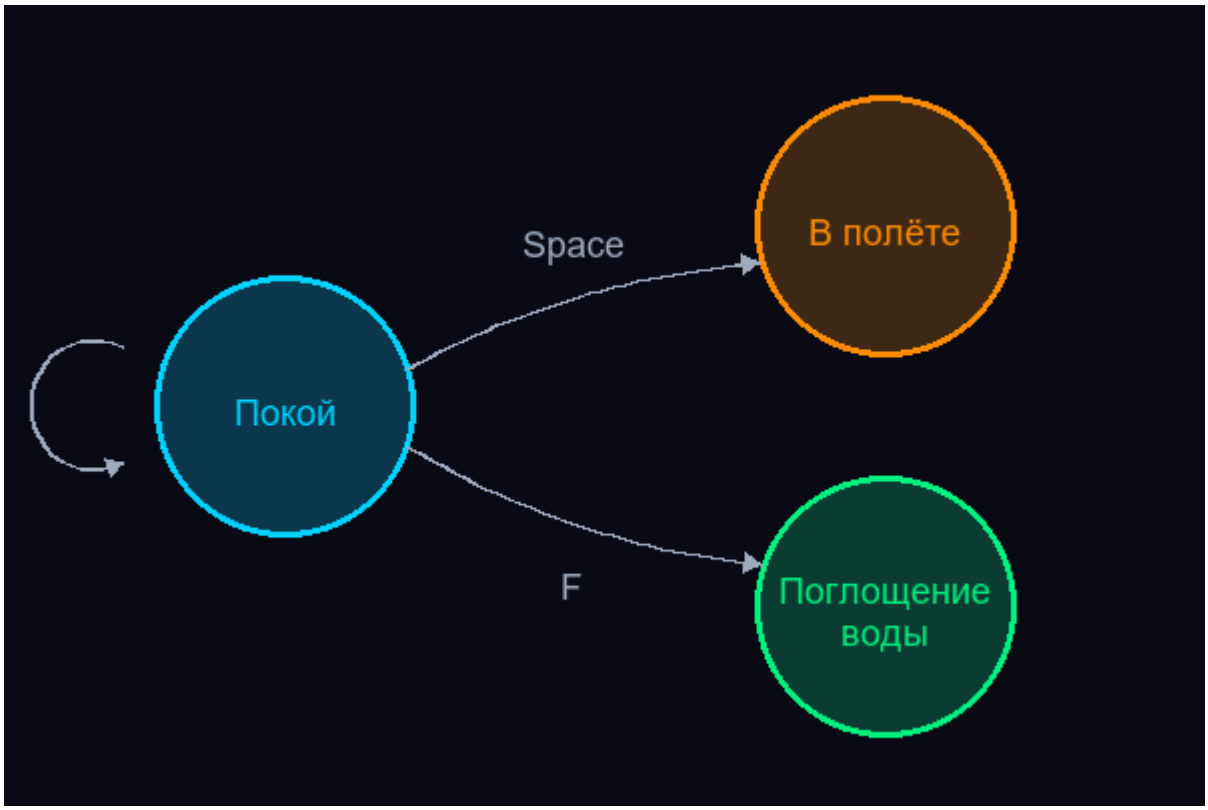
```
    is_jumping = False
    is_drinking = False
    is_attacking = False
    is_opening_door = False
```

```
5   jump_timer = 0
6   can_cancel_jump = False
7
8   while running:
9       for event in pygame.event.get():
10          if event.type == pygame.KEYDOWN:
11              if event.key == pygame.K_SPACE:
12                  if not is_jumping and not is_drinking and not is_attacking
13                      is_jumping = True
14                      can_cancel_jump = True
15                      jump_timer = 0
16              elif event.key == pygame.K_f:
17                  if not is_jumping and not is_drinking and not is_attacking
18                      is_drinking = True
19              elif event.key == pygame.K_e:
20                  if not is_jumping and not is_drinking and not is_attacking
21                      is_opening_door = True
22              elif event.key == pygame.K_LSHIFT:
23                  if not is_jumping and not is_drinking and not is_opening_d
24                      is_attacking = True
25              elif event.key == pygame.K_ESCAPE:
26                  if is_jumping and can_cancel_jump:
27                      is_jumping = False
28                      can_cancel_jump = False
29                      jump_timer = 0
30
31      if is_jumping:
32          jump_timer += 1
33          if jump_timer > 5:
34              can_cancel_jump = False  # too late to cancel
35          if jump_timer > 20:
36              is_jumping = False
37              jump_timer = 0
```

Четыре действия - уже шесть флагов и каждый `if` проверяет по четыре условия. Добавьте пр

остальные `if` -ы. Это флаговый ад.

А в виде автомата это может выглядеть так:



И причем здесь игра? При том, что наш автомат всегда находится в одном конкретном состоянии и попасть он может в определенный набор других состояний с помощью четко определенных действий, которые мы заранее определяем. Поэтому все, что нам нужно - получить состояние и передать действие. Если из нашего состояния нет стрелочки с указанным действием, значит, нам ничего не надо делать. И код может выглядеть так:

Python C#

```python
from enum import Enum, auto

class StateType(Enum):
    IDLE = auto()
    JUMPING = auto()
    DRINKING = auto()

class State:
    state_type = None  # override in subclasses

    def handle_input(self, event):
        pass

    def update(self):
        pass

    def draw(self, screen):
        pass
```

```
21 class IdleState(State):
22     state_type = StateType.IDLE
23 
24     def handle_input(self, event):
25         if event.type == pygame.KEYDOWN:
26             if event.key == pygame.K_SPACE:
27                 return JumpingState()
28             elif event.key == pygame.K_f:
29                 return DrinkingState()
30         return self
31 
32     def draw(self, screen):
33         pygame.draw.rect(screen, BLUE, (player_pos[0], player_pos[1], playe
34 
35 class JumpingState(State):
36     state_type = StateType.JUMPING
37 
38     def __init__(self):
39         self.jump_height = 20
40 
41     def update(self):
42         # ...
43         player_pos[1] -= self.jump_height
44 
45     def draw(self, screen):
46         pygame.draw.rect(screen, BLUE, (player_pos[0], player_pos[1], playe
47 
48     def handle_input(self, event):
49         # After one frame return to idle
50         return IdleState()
51 
52 class DrinkingState(State):
53     state_type = StateType.DRINKING
54 
55     def draw(self, screen):
56         pygame.draw.ellipse(screen, RED, (player_pos[0], player_pos[1], pla
57 
58     def handle_input(self, event):
59         return IdleState()
60 
61 
62 current_state = IdleState()
63 
64 running = True
65 while running:
66     for event in pygame.event.get():
67         # ...
68         current_state = current_state.handle_input(event)
69 
70     current_state.update()
       current_state.draw(screen)
```

Подобные автоматы можно строить не только для человечка, но и для других сущностей.

Менеджер сцен

Что такое сцена? Грубо говоря, набор каких-то объектов, правил взаимодействия с ними. Можно считать сцену в игре/движке той же сценой, что и в театре. Любой спектакль - набор сцен. При этом антракт мы тоже можем назвать сценой. Но она особая - мы можем выйти из зала, позвонить по телефону, при этом актеры не играют свои роли, а ведут себя как обычные люди.

Какие сцены могут быть у нас в игре?

- **Сцена главного меню** - задний фон, несколько кнопок в духе “Начать игру”, “Настройки”.
- **Сцена геймплея** - главный герой, ландшафт, задники, полоска здоровья.
- **Сцена паузы** - игра приостановлена, появляется меню на переднем плане (поверх сцены геймплея)

Обратите внимание, что сцена главного меню не содержит главного героя, следовательно, мы не можем им управлять, что логично, т.к. игра еще не началась. И тут нам тоже могут пригодиться конечные автоматы, где состояниями являются сцены. В данном примере переход от главного меню возможен по действию “Нажатие на кнопку “Начать игру”.

Дополнительные плюшки

Естественно, подобный автомат напрямую не пересекается с автоматом игрока.

Однако, возможны ситуации, когда автоматы являются вложенными . Например, можно сделать автомат для отдельной сцены "Главное меню" (состояние игры), где мы определяем переходы между различными подсценами.

Мы можем гораздо проще визуализировать все наши переплетения условий, состояний. Так нам будет проще и дебажить, и разрабатывать, и сохранять состояние игры и всех сущностей.

Но у FSM есть и минусы. Когда состояний становится много, количество переходов растёт лавинообразно - автомат превращается в паутину. Если нужны параллельные состояния (бежать **и** стрелять), одного автомата не хватит - придётся комбинировать несколько или переходить к деревьям поведений.

Что	Где подробнее
Анимации - тоже автоматы состояний	Анимации
ИИ врагов на основе FSM (Patrol/Chase/Attack)	ИИ врагов
Деревья поведений - когда FSM не хватает	Деревья поведений

Что	Где подробнее
Переходы через события вместо флагов	Событийная модель
Пошаговый цикл как FSM	Пошаговые игры
Флаговый ад - зачем вообще нужны автоматы	Типичные ошибки
FSM с вероятностными переходами	Цепи Маркова
Обзор архитектурных подходов	Архитектура

Powered by Wiki.js